

```
In [1]: #Import numerical libraries
import pandas as pd
import numpy as np

#Import graphical plotting libraries
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

#Import Linear Regression Machine Learning Libraries
from sklearn import preprocessing
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score
```

```
In [2]: data = pd.read_csv(r"C:\Users\LIKHITH\Downloads\car-mpg.csv")
data.head()
```

```
Out[2]:
```

	mpg	cyl	displacement	horsepower	weight	acceleration	year	origin	car_type	car_name
0	18.0	8	307.0	130	3504	12.0	70	1	0	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	0	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	0	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	0	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	0	ford torino

```
In [3]: data = data.drop(['car_name'], axis = 1)
```

```
In [4]: data
```

Out[4]:

	mpg	cyl	disp	hp	wt	acc	yr	origin	car_type
0	18.0	8	307.0	130	3504	12.0	70	1	0
1	15.0	8	350.0	165	3693	11.5	70	1	0
2	18.0	8	318.0	150	3436	11.0	70	1	0
3	16.0	8	304.0	150	3433	12.0	70	1	0
4	17.0	8	302.0	140	3449	10.5	70	1	0
...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86	2790	15.6	82	1	1
394	44.0	4	97.0	52	2130	24.6	82	2	1
395	32.0	4	135.0	84	2295	11.6	82	1	1
396	28.0	4	120.0	79	2625	18.6	82	1	1
397	31.0	4	119.0	82	2720	19.4	82	1	1

398 rows × 9 columns

```

In [8]: # Replace '?' with NaN
data = data.replace('?', np.nan)

# Convert numeric columns safely
for col in data.columns:
    try:
        data[col] = pd.to_numeric(data[col])
    except:
        pass

# Encode origin ONLY if it exists
if 'origin' in data.columns:
    data['origin'] = data['origin'].map({1:'america', 2:'europe', 3:'asia'})
    data = pd.get_dummies(data, columns=['origin'])

# Fill NaN ONLY in numeric columns
num_cols = data.select_dtypes(include=np.number).columns
data[num_cols] = data[num_cols].fillna(data[num_cols].median())

```

In [9]: data

Out[9]:

	mpg	cyl	disp	hp	wt	acc	yr	car_type	origin_america	origin_asia	origin_e
0	18.0	8	307.0	130.0	3504	12.0	70	0	True	False	
1	15.0	8	350.0	165.0	3693	11.5	70	0	True	False	
2	18.0	8	318.0	150.0	3436	11.0	70	0	True	False	
3	16.0	8	304.0	150.0	3433	12.0	70	0	True	False	
4	17.0	8	302.0	140.0	3449	10.5	70	0	True	False	
...	...	...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86.0	2790	15.6	82	1	True	False	
394	44.0	4	97.0	52.0	2130	24.6	82	1	False	False	
395	32.0	4	135.0	84.0	2295	11.6	82	1	True	False	
396	28.0	4	120.0	79.0	2625	18.6	82	1	True	False	
397	31.0	4	119.0	82.0	2720	19.4	82	1	True	False	

398 rows × 11 columns



model building

```
In [14]: X = data.drop(['mpg'], axis = 1) # independent variable
         y = data[['mpg']] #dependent variable
```

```
In [20]: y
```

Out[20]:

	mpg
0	18.0
1	15.0
2	18.0
3	16.0
4	17.0
...	...
393	27.0
394	44.0
395	32.0
396	28.0
397	31.0

398 rows × 1 columns

In [18]: X

Out[18]:

	cyl	disp	hp	wt	acc	yr	car_type	origin_america	origin_asia	origin_europe
0	8	307.0	130.0	3504	12.0	70	0	True	False	False
1	8	350.0	165.0	3693	11.5	70	0	True	False	False
2	8	318.0	150.0	3436	11.0	70	0	True	False	False
3	8	304.0	150.0	3433	12.0	70	0	True	False	False
4	8	302.0	140.0	3449	10.5	70	0	True	False	False
...	...	...	...	...	...	...	...	...	...	...
393	4	140.0	86.0	2790	15.6	82	1	True	False	False
394	4	97.0	52.0	2130	24.6	82	1	False	False	True
395	4	135.0	84.0	2295	11.6	82	1	True	False	False
396	4	120.0	79.0	2625	18.6	82	1	True	False	False
397	4	119.0	82.0	2720	19.4	82	1	True	False	False

398 rows × 10 columns

In [21]: *#Scaling the data*

X_s = preprocessing.scale(X)

```
X_s = pd.DataFrame(X_s, columns = X.columns) #converting scaled data into dataframe  
y_s = preprocessing.scale(y)  
y_s = pd.DataFrame(y_s, columns = y.columns) #ideally train, test data should be in
```

In [22]: X_s

Out[22]:

	cyl	displacement	horsepower	weight	acceleration	year	car_type	origin_ame
0	1.498191	1.090604	0.673118	0.630870	-1.295498	-1.627426	-1.062235	0.773
1	1.498191	1.503514	1.589958	0.854333	-1.477038	-1.627426	-1.062235	0.773
2	1.498191	1.196232	1.197027	0.550470	-1.658577	-1.627426	-1.062235	0.773
3	1.498191	1.061796	1.197027	0.546923	-1.295498	-1.627426	-1.062235	0.773
4	1.498191	1.042591	0.935072	0.565841	-1.840117	-1.627426	-1.062235	0.773
...	...	...	...	...	...	...	...	...
393	-0.856321	-0.513026	-0.479482	-0.213324	0.011586	1.621983	0.941412	0.773
394	-0.856321	-0.925936	-1.370127	-0.993671	3.279296	1.621983	0.941412	-1.292
395	-0.856321	-0.561039	-0.531873	-0.798585	-1.440730	1.621983	0.941412	0.773
396	-0.856321	-0.705077	-0.662850	-0.408411	1.100822	1.621983	0.941412	0.773
397	-0.856321	-0.714680	-0.584264	-0.296088	1.391285	1.621983	0.941412	0.773

398 rows × 10 columns



In [23]: y_s

Out[23]:

	mpg
0	-0.706439
1	-1.090751
2	-0.706439
3	-0.962647
4	-0.834543
...	...
393	0.446497
394	2.624265
395	1.087017
396	0.574601
397	0.958913

398 rows × 1 columns

In [12]: *#Split into train, test set*

```
X_train, X_test, y_train, y_test = train_test_split(X_s, y_s, test_size = 0.30, rand  
X_train.shape
```

Out[12]: (278, 10)

simple linear model

In [24]: *#Fit simple linear model and find coefficients*

```
regression_model = LinearRegression()  
regression_model.fit(X_train, y_train)  
  
for idx, col_name in enumerate(X_train.columns):  
    print('The coefficient for {} is {}'.format(col_name, regression_model.coef_[0])  
  
intercept = regression_model.intercept_[0]  
print('The intercept is {}'.format(intercept))
```

The coefficient for cyl is 0.321022385691611
 The coefficient for disp is 0.32483430918483897
 The coefficient for hp is -0.22916950059437569
 The coefficient for wt is -0.7112101905072298
 The coefficient for acc is 0.014713682764191237
 The coefficient for yr is 0.3755811949510748
 The coefficient for car_type is 0.3814769484233099
 The coefficient for origin_america is -0.07472247547584178
 The coefficient for origin_asia is 0.044515252035677896
 The coefficient for origin_europe is 0.04834854953945386
 The intercept is 0.019284116103639764

regularized ridge regression

In [25]: *#alpha factor here is Lambda (penalty term) which helps to reduce the magnitude of*

```

ridge_model = Ridge(alpha = 0.3)
ridge_model.fit(X_train, y_train)

print('Ridge model coef: {}'.format(ridge_model.coef_))
#As the data has 10 columns hence 10 coefficients appear here

```

Ridge model coef: [0.31649043 0.31320707 -0.22876025 -0.70109447 0.01295851 0.37447352
 0.37725608 -0.07423624 0.04441039 0.04784031]

regularized lasso regression

In [26]: *#alpha factor here is Lambda (penalty term) which helps to reduce the magnitude of*

```

lasso_model = Lasso(alpha = 0.1)
lasso_model.fit(X_train, y_train)

print('Lasso model coef: {}'.format(lasso_model.coef_))
#As the data has 10 columns hence 10 coefficients appear here

```

Lasso model coef: [-0. -0. -0.01690287 -0.51890013 0. 0.28138241
 0.1278489 -0.01642647 0. 0.]

score comparison

In [27]: *#Model score - r^2 or coeff of determinant*
#r^2 = 1-(RSS/TSS) = Regression error/TSS

```

#Simple Linear Model
print(regression_model.score(X_train, y_train))
print(regression_model.score(X_test, y_test))

print('*****')

```

```
#Ridge
print(ridge_model.score(X_train, y_train))
print(ridge_model.score(X_test, y_test))

print('*****')
#Lasso
print(lasso_model.score(X_train, y_train))
print(lasso_model.score(X_test, y_test))
```

```
0.8343770256960538
0.8513421387780066
*****
0.8343617931312616
0.8518882171608507
*****
0.7938010766228453
0.8375229615977083
```

model parameter tuning

```
In [29]: data_train_test = pd.concat([X_train, y_train], axis =1)
data_train_test.head()
```

```
Out[29]:
```

	cyl	dis	hp	wt	acc	yr	car_type	origin_ame
350	-0.856321	-0.849116	-1.081977	-0.893172	-0.242570	1.351199	0.941412	0.773
59	-0.856321	-0.925936	-1.317736	-0.847061	2.879909	-1.085858	0.941412	-1.292
120	-0.856321	-0.695475	0.201600	-0.121101	-0.024722	-0.815074	0.941412	-1.292
12	1.498191	1.983643	1.197027	0.934732	-2.203196	-1.627426	-1.062235	0.773
349	-0.856321	-0.983552	-0.951000	-1.165111	0.156817	1.351199	0.941412	-1.292

```
In [30]: import statsmodels.formula.api as smf
ols1 = smf.ols(formula = 'mpg ~ cyl+disp+hp+wt+acc+yr+car_type+origin_america+origi
ols1.params
```

```
Out[30]: Intercept      0.019284
cyl      0.321022
dis      0.324834
hp      -0.229170
wt      -0.711210
acc      0.014714
yr      0.375581
car_type  0.381477
origin_america -0.074722
origin_europe  0.048349
origin_asia  0.044515
dtype: float64
```

```
In [31]: print(ols1.summary())
```

OLS Regression Results

=====						
Dep. Variable:	mpg	R-squared:	0.834			
Model:	OLS	Adj. R-squared:	0.829			
Method:	Least Squares	F-statistic:	150.0			
Date:	Thu, 18 Dec 2025	Prob (F-statistic):	3.12e-99			
Time:	11:23:35	Log-Likelihood:	-146.89			
No. Observations:	278	AIC:	313.8			
Df Residuals:	268	BIC:	350.1			
Df Model:	9					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

Intercept	0.0193	0.025	0.765	0.445	-0.030	0.069
cyl	0.3210	0.112	2.856	0.005	0.100	0.542
displacement	0.3248	0.128	2.544	0.012	0.073	0.576
hp	-0.2292	0.079	-2.915	0.004	-0.384	-0.074
wt	-0.7112	0.088	-8.118	0.000	-0.884	-0.539
acc	0.0147	0.039	0.373	0.709	-0.063	0.092
yr	0.3756	0.029	13.088	0.000	0.319	0.432
car_type	0.3815	0.067	5.728	0.000	0.250	0.513
origin_america	-0.0747	0.020	-3.723	0.000	-0.114	-0.035
origin_europe	0.0483	0.021	2.270	0.024	0.006	0.090
origin_asia	0.0445	0.020	2.175	0.031	0.004	0.085
=====						
Omnibus:	22.678	Durbin-Watson:	2.105			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	36.139			
Skew:	0.513	Prob(JB):	1.42e-08			
Kurtosis:	4.438	Cond. No.	1.59e+16			
=====						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 6.14e-30. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

```
In [32]: # Lets check Sum of Squared Errors (SSE) by predicting value of y for test cases and
mse = np.mean((regression_model.predict(X_test)-y_test)**2)

# root of mean_sq_error is standard deviation i.e. avg variance between predicted and actual
import math
rmse = math.sqrt(mse)
print('Root Mean Squared Error: {}'.format(rmse))
```

Root Mean Squared Error: 0.37766934254087847

So there is an avg. mpg difference of 0.37 from real mpg

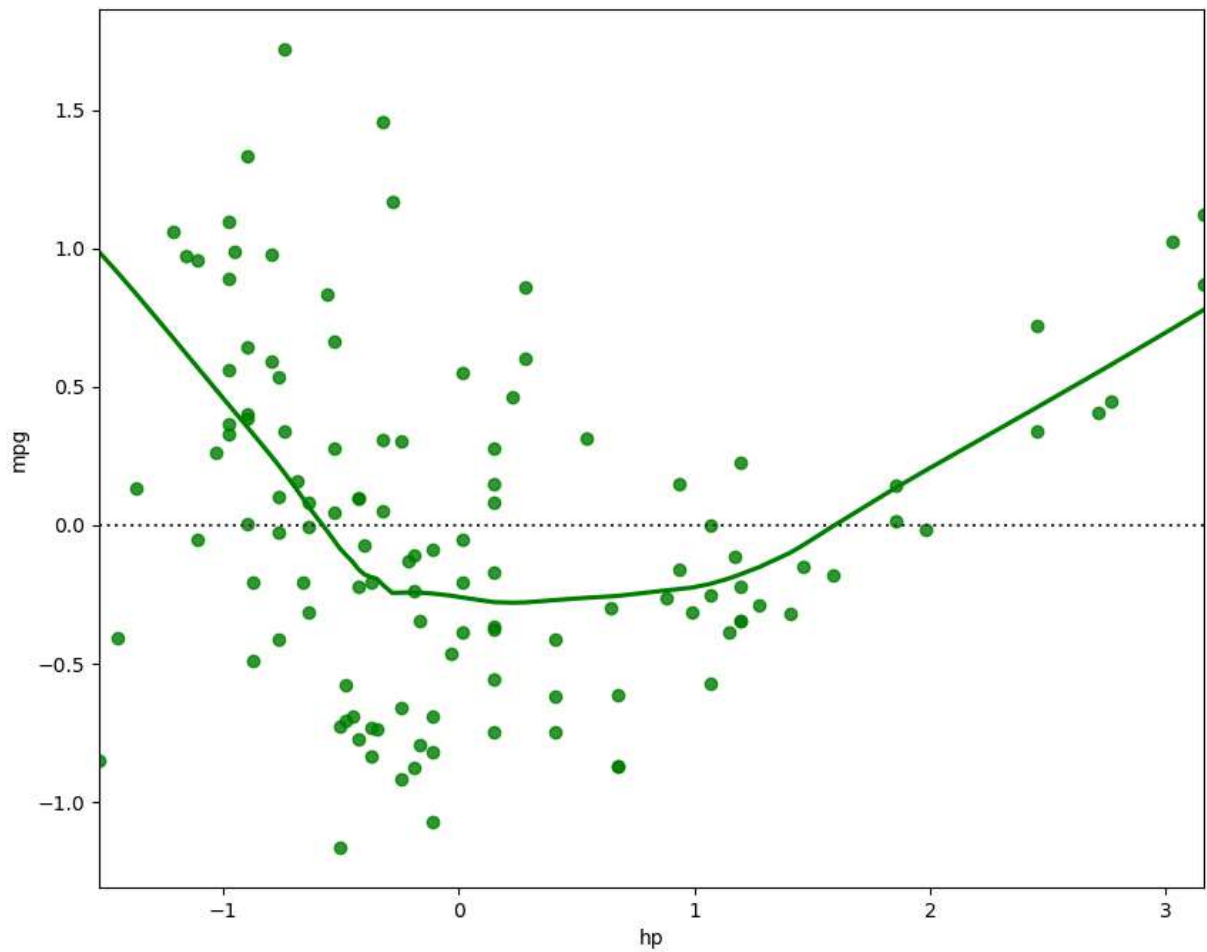
```
In [34]: # Is OLS a good model ? Lets check the residuals for some of these predictor.

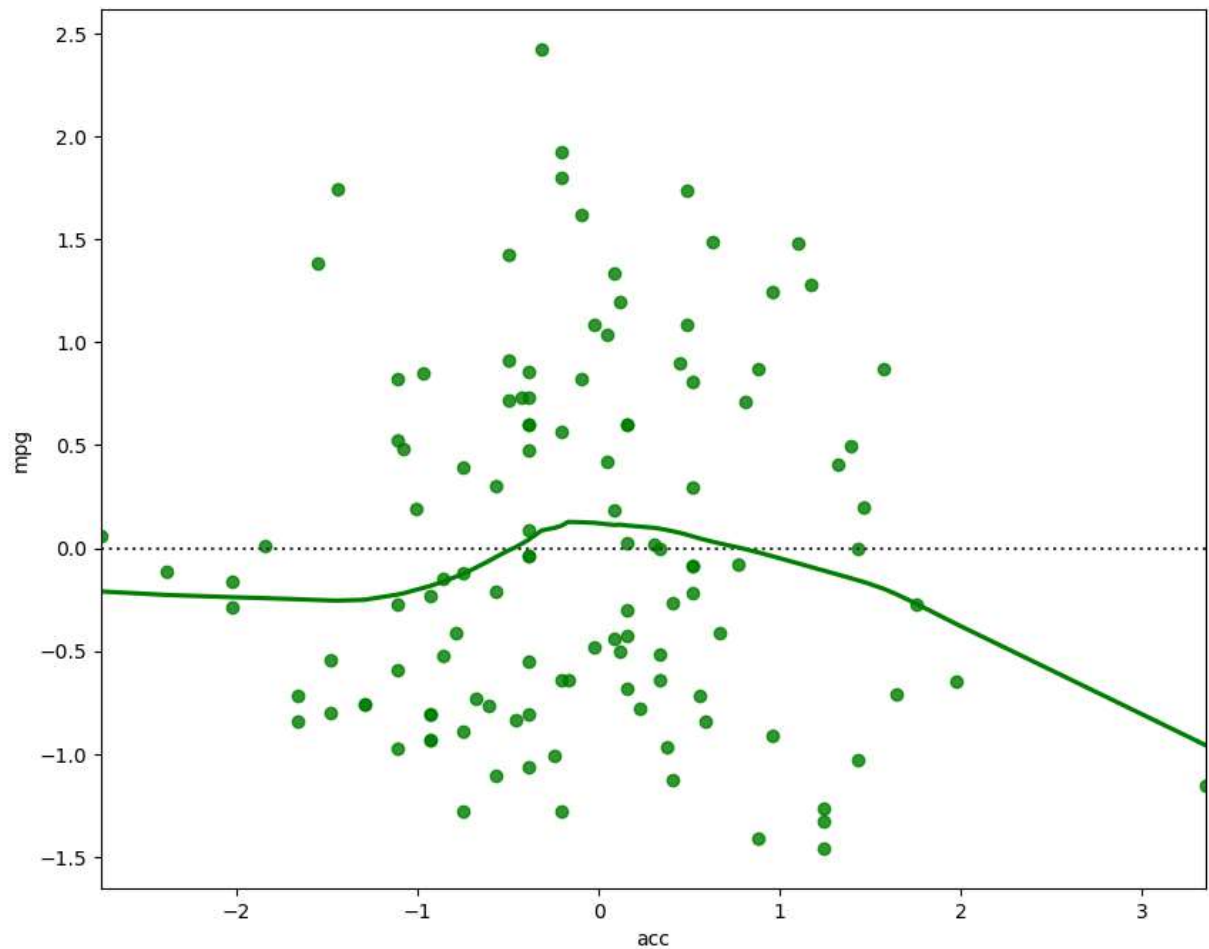
fig = plt.figure(figsize=(10,8))
sns.residplot(x= X_test['hp'], y= y_test['mpg'], color='green', lowess=True )
```

```
fig = plt.figure(figsize=(10,8))  
sns.residplot(x= X_test['acc'], y= y_test['mpg'], color='green', lowess=True )
```

Out[34]: <Axes: xlabel='acc', ylabel='mpg'>

In [35]: plt.show()



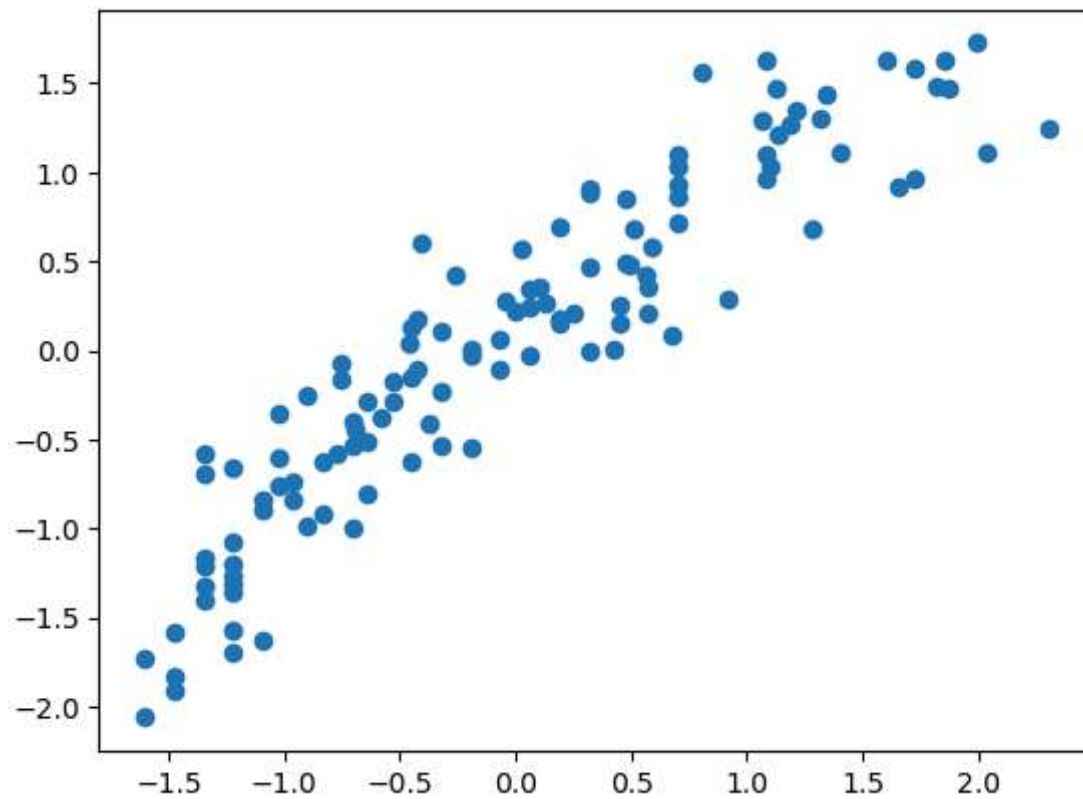


```
In [36]: # predict mileage (mpg) for a set of attributes not in the training or test set
y_pred = regression_model.predict(X_test)

# Since this is regression, plot the predicted y value vs actual y values for the t
# A good model's prediction will be close to actual leading to high R and R2 values
#plt.rcParams['figure.dpi'] = 500
plt.scatter(y_test['mpg'], y_pred)
```

```
Out[36]: <matplotlib.collections.PathCollection at 0x23c74dded50>
```

```
In [37]: plt.show()
```



In []: